

"""

Inarticulation Function – G Translation.

The Inarticulation Function G is the final stage of the Semantic Engine: it takes the ****solved**** State-Space Closure (Ω) and translates it back into tangible production metrics.

In the UMA framework:

$G: (z_\Omega, \text{posterior}_\Omega, \text{friction_summary}, \text{constraint_report})$
→ ProductionMetrics

where z_Ω is the closed projected field state, posterior_Ω is the final posterior, and the summaries carry the Semantic Integral and constraint diagnostics.

This is the inverse of the initial ingestion step: the engine "inarticulates" (makes concrete) the abstract solved trajectory as specific production outputs.

Two built-in translators:

NullInarticulator:

Identity pass – returns the raw solved quantities with no extra transformation. Useful for debugging and unit tests.

ProjectionInarticulator:

Applies a user-supplied linear readout $W : \mathbb{R}^d \rightarrow \mathbb{R}^m$ to z_Ω , optionally passed through a user-supplied nonlinearity ϕ . Interprets the readout as production signal coordinates: semantic density, trajectory delta, scaling coefficient.

LLMInarticulator:

Wraps `uma.observations.llm.LLMOperator`. After closure, uses the calibrated LLM observation model to "read out" the production signal $y = \text{llm_op}(z_\Omega)$, expressed as production metrics.

ProductionMetrics dataclass carries all production output:

semantic_density: 1 - SF_total (the "fill" of the solved space)
trajectory_delta: $\|z_\Omega - z_{\text{target}}\|$ (residual after closure)
scaling_coefficient: $\det(\Sigma_\Omega)^{-1/d}$ (information concentration)
stability_index: 1 - final_violation / (initial_violation + ϵ)
readout: $\mathbb{R}^m$ vector (Inarticulator-specific production signal)
stage: 'ingestion' | 'stability' | 'inarticulation' | 'solve'
closed: whether Ω was achieved

The `stage` field mirrors the four-stage table from the Semantic Engine spec:

Ingestion	→ Initial differential, friction at $t=0$
Stability	→ Equalities Filter, friction descending
Inarticulation	→ G Translation, friction below threshold
Solve	→ State-Space Closure, closed=True

"""

```
from __future__ import annotations
import numpy as np
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Optional, Callable, Dict, Any, List
```

```
# -----
# ProductionMetrics – the output of G
# -----
```

```
@dataclass
```

```
class ProductionMetrics:
```

```
    """
```

```
    Tangible production output from the Inarticulation stage.
```

```
    All quantities are scalar or low-dimensional; they are intended as
    direct inputs to downstream production processes (scaling decisions,
    architecture routing, operator alerts, etc.).
```

```
    """
```

```
    # Core semantic quantities
```

```
    semantic_density: float          #  $1 - SF_{total} \in [0, 1]$ 
```

```
    trajectory_delta: float          #  $\|z_{\Omega} - z_{target}\|$  (absolute)
```

```
    scaling_coefficient: float        #  $\det(\Sigma_{\Omega})^{-1/d}$  – information concentratio
```

```
    # Constraint health
```

```
    stability_index: float            #  $1 - final\_constraint\_violation$ 
```

```
    constraint_converged: bool         # whether ConstraintSet achieved tol
```

```
    # Stage classification
```

```
    stage: str                        # 'ingestion'|'stability'|'inarticulation'|'
```

```
    closed: bool                      # True iff  $\Omega$  achieved
```

```
    # Inarticulator-specific readout
```

```
    readout: Optional[np.ndarray] = None    # (m,) production signal
```

```
    readout_labels: Optional[List[str]] = None
```

```
    # Full diagnostic passthrough
```

```
    friction_summary: Optional[Dict[str, Any]] = None
```

```

constraint_report: Optional[Dict[str, Any]] = None

def as_dict(self) -> Dict[str, Any]:
    d = {
        "semantic_density": round(self.semantic_density, 6),
        "trajectory_delta": round(self.trajectory_delta, 6),
        "scaling_coefficient": round(self.scaling_coefficient, 6),
        "stability_index": round(self.stability_index, 6),
        "constraint_converged": self.constraint_converged,
        "stage": self.stage,
        "closed": self.closed,
    }
    if self.readout is not None:
        d["readout"] = self.readout.tolist()
    if self.readout_labels is not None:
        d["readout_labels"] = self.readout_labels
    if self.friction_summary is not None:
        d["friction_summary"] = self.friction_summary
    if self.constraint_report is not None:
        d["constraint_report"] = self.constraint_report
    return d

def __repr__(self) -> str:
    return (
        f"ProductionMetrics(stage={self.stage!r}, closed={self.closed}, "
        f"density={self.semantic_density:.4f}, "
        f"delta={self.trajectory_delta:.4f}, "
        f"stability={self.stability_index:.4f})"
    )

# -----
# Stage classifier
# -----

def classify_stage(sf_value: float, is_closed: bool,
                  sf_stability_threshold: float = 0.50,
                  sf_inarticulation_threshold: float = 0.20) -> str:
    """
    Map the current SF_total to the four-stage production pipeline.

    sf_value = 1.0 → Ingestion      (raw, high-friction)
    sf_value ∈ (0.5, 1.0) → Stability (filter stabilizing)
    sf_value ∈ (0.2, 0.5) → Inarticulation (translating to output)
    closed = True → Solve          (state-space closure achieved)
    """
    if is_closed:

```

```

        return "solve"
    if sf_value > sf_stability_threshold:
        return "ingestion"
    if sf_value > sf_inarticulation_threshold:
        return "stability"
    return "inarticulation"

# -----
# Scaling coefficient helper
# -----

def _scaling_coefficient(posterior) -> float:
    """
    Compute  $\det(\Sigma)^{-1/d}$  – the information concentration.
    High value = tightly concentrated posterior (low entropy).
    Low value = diffuse posterior (high entropy).
    """
    from uma.core.state import FieldPosterior
    d = posterior.dim
    if posterior.representation == "full":
        try:
            sign, logdet = np.linalg.slogdet(posterior.Sigma)
            if sign <= 0:
                return 0.0
            return float(np.exp(-logdet / d))
        except Exception:
            return 0.0
    elif posterior.representation == "lowrank":
        L = posterior.L
        _, sv, _ = np.linalg.svd(L, full_matrices=False)
        sv = np.maximum(sv, 1e-15)
        log_det = 2 * np.sum(np.log(sv))
        return float(np.exp(-log_det / d))
    elif posterior.representation in ("ensemble", "particle"):
        z_mean = posterior.mean
        if posterior.representation == "ensemble":
            X = posterior.ensemble - z_mean
            n = X.shape[0]
            S = (X.T @ X) / max(n - 1, 1)
        else:
            w = np.exp(posterior.log_weights - posterior.log_weights.max())
            w /= w.sum()
            X = posterior.particles - z_mean
            S = (w[:, None, None] * X[:, :, None] * X[:, None, :]).sum(0)
        try:
            sign, logdet = np.linalg.slogdet(S)

```

```

        if sign <= 0:
            return 0.0
        return float(np.exp(-logdet / d))
    except Exception:
        return 0.0
    return 0.0

# -----
# Abstract Inarticulator
# -----

class Inarticulator(ABC):
    """Abstract base for G Translation functions."""

    @abstractmethod
    def translate(
        self,
        z_omega: np.ndarray,
        posterior,
        z_target: np.ndarray,
        friction_summary: Dict[str, Any],
        constraint_report: Dict[str, Any],
    ) -> ProductionMetrics:
        """Translate closed state to ProductionMetrics."""

# -----
# NullInarticulator
# -----

class NullInarticulator(Inarticulator):
    """
    Identity pass – returns raw metrics with no additional transformation.
    Useful for debugging and as a default when no readout model is specified.
    """

    def translate(self, z_omega, posterior, z_target, friction_summary,
                  constraint_report) -> ProductionMetrics:
        sf_total = friction_summary.get("sf_total", 1.0)
        is_closed = friction_summary.get("is_closed", False)
        stage = classify_stage(sf_total, is_closed)
        delta = float(np.linalg.norm(z_omega - z_target))
        scl = _scaling_coefficient(posterior)
        final_viol = constraint_report.get("final_total_violation", 0.0)
        converged = constraint_report.get("converged", True)

```

```

    stability = float(np.clip(1.0 - final_viol, 0.0, 1.0))

    return ProductionMetrics(
        semantic_density=1.0 - sf_total,
        trajectory_delta=delta,
        scaling_coefficient=scl,
        stability_index=stability,
        constraint_converged=converged,
        stage=stage,
        closed=is_closed,
        readout=z_omega.copy(),
        readout_labels=[f"z_{i}" for i in range(len(z_omega))],
        friction_summary=friction_summary,
        constraint_report=constraint_report,
    )

# -----
# ProjectionInarticulator
# -----

class ProjectionInarticulator(Inarticulator):
    """
    Linear readout  $W : \mathbb{R}^d \rightarrow \mathbb{R}^m$  applied to  $z_\Omega$ , then  $\phi$  nonlinearity.

    The readout coordinates are interpreted as production signal:
    by convention the first three dimensions (if  $m \geq 3$ ) carry:
        [0] semantic density proxy (high  $\rightarrow$  dense, well-solved)
        [1] trajectory correction delta proxy
        [2] scaling target proxy

    All three are renormalized against  $z_{\text{target}}$  to produce dimensionless
    production metrics.
    """

    def __init__(
        self,
        W: np.ndarray,
        phi: Optional[Callable[[np.ndarray], np.ndarray]] = None,
        labels: Optional[List[str]] = None,
    ):
        """
        W:          (m, d) readout matrix.
        phi:        Optional element-wise nonlinearity (default identity).
        labels:     Optional names for readout dimensions.
        """
        self._W = np.asarray(W, dtype=float)

```

```

        self._phi = phi if phi is not None else lambda x: x
        self._labels = labels

def translate(self, z_omega, posterior, z_target, friction_summary,
              constraint_report) -> ProductionMetrics:
    sf_total = friction_summary.get("sf_total", 1.0)
    is_closed = friction_summary.get("is_closed", False)
    stage = classify_stage(sf_total, is_closed)

    readout_raw = self._phi(self.W @ z_omega)
    delta = float(np.linalg.norm(z_omega - z_target))
    scl = _scaling_coefficient(posterior)
    final_viol = constraint_report.get("final_total_violation", 0.0)
    converged = constraint_report.get("converged", True)
    stability = float(np.clip(1.0 - final_viol, 0.0, 1.0))

    return ProductionMetrics(
        semantic_density=1.0 - sf_total,
        trajectory_delta=delta,
        scaling_coefficient=scl,
        stability_index=stability,
        constraint_converged=converged,
        stage=stage,
        closed=is_closed,
        readout=readout_raw,
        readout_labels=self._labels,
        friction_summary=friction_summary,
        constraint_report=constraint_report,
    )

@classmethod
def principal_components(cls, dim: int, m: int = 4,
                        labels: Optional[List[str]] = None) -> "ProjectionI
    """
    Convenience: random orthonormal readout basis of rank m.
    In production, replace with a calibrated W from domain data.
    """
    rng = np.random.default_rng(0)
    A = rng.standard_normal((dim, m))
    W, _ = np.linalg.qr(A)
    return cls(W=W.T[:m, :].copy(), labels=labels)

```